

FSI

Práctica 2.1

Manipulación de imágenes con Matlab

20 de octubre de 2024

Mario Medrano Paredes, Alonso Sandoval Martínez

Contenido

1. Generación de una máscara	3
2. Generación de una carta de colores	5
3. Carta de colores modificada	6
4. Cuantificación con y sin dithering.....	7
5. Imágenes cuantificadas y enmascaradas.....	8
6. Cuantificación de imagen enmascarada	9

1. Generación de una máscara

Genere y represente una imagen binaria de tamaño 256×256 correspondiente a la máscara que se muestra en la figura 1. Los radios de la corona circular deben ser 32 y 128 píxeles para las circunferencias interior y exterior respectivamente.

Para lograr conseguir esto hemos seguido el siguiente procedimiento. Primero, definimos el tamaño de la matriz cuadrada, que será de 256×256 . Siendo `radio_a` el radio interior y `radio_b` el radio exterior, estos valores determinan los límites entre los cuales la máscara circular será blanca:

```
size = 256;  
radio_a = 32;  
radio_b = 128;
```

Se crea la matriz `mask` de 256×256 . Inicialmente cada píxel tendrá valor 0 (negro), y más adelante se actualizarán los valores a 1 (blanco). Calculamos el centro de la imagen dividiendo el tamaño de esta entre 2, desde este punto mediremos la distancia a cada píxel para determinar si está dentro o fuera de la corona circular:

```
mask = zeros(size);  
center = size / 2;
```

Utilizamos un doble bucle `for` para recorrer toda la matriz. Para cada píxel, se calcula la distancia al centro de la imagen y luego se evalúa si está contenida entre los radios, en caso afirmativo el píxel se pone a uno:

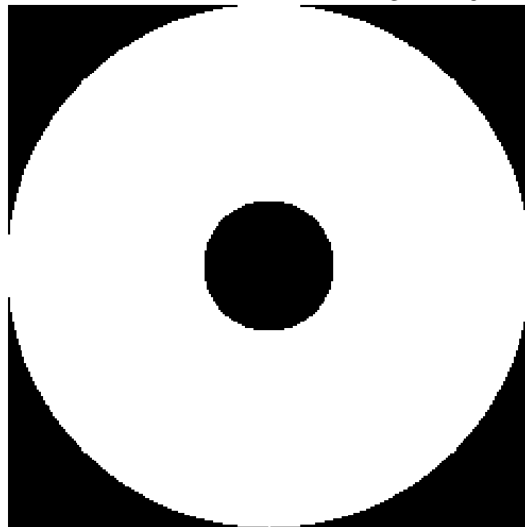
```
for x = 1:size  
    for y = 1:size  
        dist = sqrt((x - center)^2 + (y - center)^2);  
        if dist >= radio_a && dist <= radio_b  
            mask(x, y) = 1;  
        end  
    end  
end
```

Finalmente mostramos la matriz aplicándole un mapa de color “gray”, que representa la imagen en escala de grises:

```
imshow(mask);  
colormap(gray);  
title('Máscara binaria con radios 32 y 128 píxeles');
```

A continuación, podemos observar la imagen generada para la máscara:

Máscara binaria con radios 32 y 128 píxeles



2. Generación de una carta de colores

Genere ahora una carta de colores 256×256 como la mostrada en la figura 2. Para ello, debe trabajar en el espacio HSV. El canal de tinte variará en diagonal para todo su rango siguiendo la estructura de una matriz de Toeplitz (comando `toeplitz` de MATLAB). Luminancia y saturación se mantendrán constantes en su valor máximo.

Volvemos a definir el tamaño de la matriz cuadrada de 256×256 . Ahora con `linspace` creamos un vector con size también 256 que variará linealmente desde 1 hasta 0, correspondiendo a un tinte de 360 grados en HSV (rojo) hasta uno de 0 grados (también rojo), los valores intermedios cubren todo el rango de tonalidades intermedias.

Luego, creamos una matriz de toeplitz y en hue, con constantes en las diagonales, en este caso el resultado es una matriz de 256×256 donde el tinte varía en diagonal:

```
size = 256;  
hue = toeplitz(linspace(1, 0, size));
```

Ahora definimos los canales de saturación y de valor, ambas con una matriz de unos, lo que implica que la saturación y el valor son máximos en todos los píxeles. Esto garantiza colores completamente saturados y máxima luminosidad en todos los canales:

```
saturation = ones(size);  
value = ones(size);
```

A continuación, concatenamos los tres canales usando `cat`, formando una imagen HSV. Luego, convertimos la imagen de HSV a RGB para visualizarla correctamente:

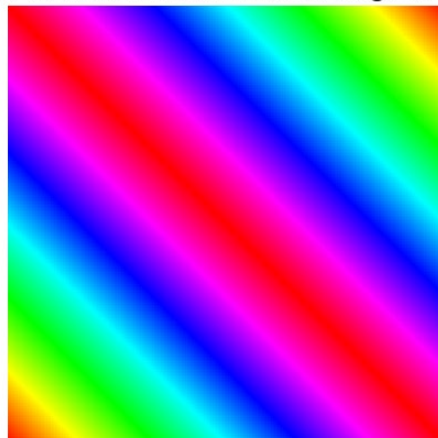
```
hsv_img = cat(3, hue, saturation, value);  
rgb_img = hsv2rgb(hsv_img);
```

Por último visualizamos la imagen ya en RGB:

```
imshow(rgb_img);  
title('Carta de colores HSV con variación diagonal del tinte');
```

La imagen resultante puede verse a continuación:

Carta de colores HSV con variación diagonal del tinte



3. Carta de colores modificada

Repita el ejercicio anterior, pero haga variar el canal de saturación con el mismo patrón que el de tinte, aunque en este caso el rango variará en el intervalo [0.5, 1] (ver figura 3). Represente la imagen y comente las diferencias obtenidas.

Este apartado es igual que el anterior, sólo que ahora modificamos la saturación de manera que ya no es máxima en todo momento:

```
saturation = 0.5 + 0.5 * toeplitz(linspace(0, 1, size));
```

Con linspace generaremos un vector de valores espaciados uniformemente desde 0 hasta 1, es decir, valores de saturación máxima y saturación mínima (gris). Luego convertimos este vector a una matriz de Toeplitz, explicada anteriormente.

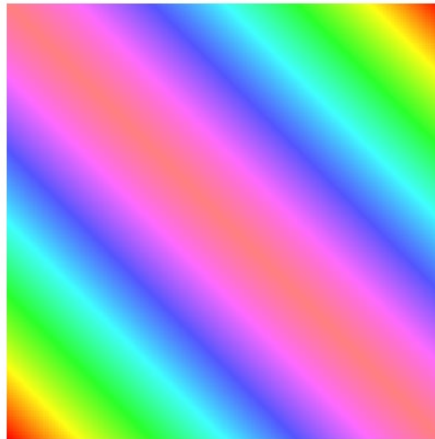
Por último, se aplica la operación $0.5 + 0.5 * (...)$ para ajustar los valores de saturación a un rango entre 0.5 y 1, es decir, cuando la matriz sea 0 en la matriz de Toeplitz, el valor mínimo de la saturación será 0.5.

Al realizar esa operación, estamos desplazando y escalando el rango de saturación, de manera que garantizamos que todos los píxeles tengan al menos un 50% de saturación para evitar valores de saturación muy bajos.

El resto del ejercicio se mantiene igual al apartado 2.

El resultado es la misma imagen de la carta de colores que en el caso anterior, pero en este caso también con una variación diagonal del valor de saturación:

Carta con variación diagonal del tinte y saturación



4. Cuantificación con y sin dithering

Utilice la orden `rgb2ind` para cuantificar la última imagen obtenida con una profundidad de 3 bits. Utilice las opciones del comando para aplicar una cuantificación de mínima varianza y aplíquela con y sin dithering. Explique las diferencias obtenidas comparando con la imagen original.

Lo primero que hacemos es convertir una imagen RGB en una imagen indexada con 2^3 colores con la función `rgb2ind`. El parámetro `'nodither'` indica que no se aplicará dithering, lo que significa que no habrá dispersión de los errores de cuantización para suavizar los bordes o las transiciones entre colores. `Map_no_dither` es el mapa de colores que contiene los 8 colores de la imagen indexada.

Mostramos esta imagen indexada sin dithering en la primera posición de una cuadrícula de gráficos de 1 fila y 2 columnas para compararla con la imagen cuantificada aplicando dithering. Aplicamos el mapa de colores antes mencionado:

```
[indexed_image_no_dither, map_no_dither] = rgb2ind(rgb_img, 8, 'nodither');  
  
subplot(1, 2, 1);  
imshow(indexed_image_no_dither, map_no_dither);  
title('Imagen cuantificada (3 bits) sin dithering');
```

A continuación haremos el proceso análogo pero con dithering. Para ello, usamos el parámetro `'dither'` en la función `rgb2ind`, lo cual nos devolverá la misma imagen que antes, pero con dithering, posteriormente la representaremos de la misma manera que en el caso anterior.

```
[indexed_image_dither, map_dither] = rgb2ind(rgb_img, 8, 'dither');  
  
subplot(1, 2, 2);  
imshow(indexed_image_dither, map_dither);  
title('Imagen cuantificada (3 bits) con dithering');
```

El resultado puede verse a continuación:

Imagen cuantificada (3 bits) con dithering

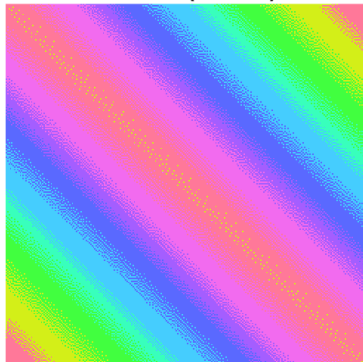


Imagen cuantificada (3 bits) sin dithering



5. Imágenes cuantificadas y enmascaradas

Aplique la máscara obtenida en el apartado 1 a las imágenes cuantificadas, de forma que sólo se visualice la región correspondiente a la corona circular. La figura 4 muestra un ejemplo para la imagen sin cuantificar.

Creemos una copia de la imagen cuantificada sin dithering en una nueva variable. Luego aplicamos la máscara creada en el apartado 1 gracias a `~mask`, haciendo que todos los píxeles que sean 0 en la máscara sean cero también en la imagen enmascarada, es decir, negro.

Creemos otra copia de la imagen cuantificada con dithering, y le aplicamos la máscara de la misma manera que a la anterior:

```
masked_image_no_dither = indexed_image_no_dither;
masked_image_no_dither(~mask) = 0;
masked_image_dither = indexed_image_dither;
masked_image_dither(~mask) = 0;
```

Por último, creamos el subplot para poder mostrar las dos imágenes enmascaradas una al lado de la otra, y las visualizamos con sus respectivos títulos:

```
subplot(1, 2, 1);
imshow(masked_image_no_dither, map_no_dither);
title('Imagen cuantificada sin dithering enmascarada');

subplot(1, 2, 2);
imshow(masked_image_dither, map_dither);
title('Imagen cuantificada con dithering enmascarada');
```

Finalmente, obtenemos las siguientes imágenes:

Imagen cuantificada sin dithering enmascarada

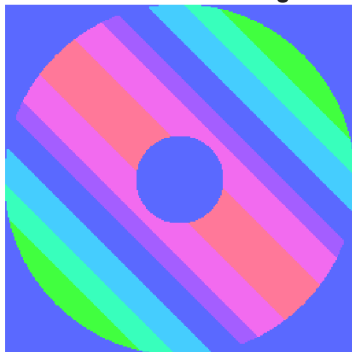
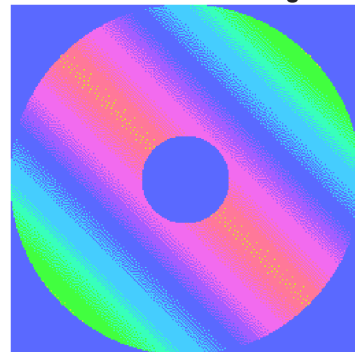


Imagen cuantificada con dithering enmascarada



6. Cuantificación de imagen enmascarada

Repita el ejercicio 4 para la imagen enmascarada.

Primero aplicamos la máscara a la imagen original, esta vez, con la función `bsxfun`. Esta función aplica una operación de manera vectorizada entre matrices del mismo tamaño, en este caso, cada pixel de `rgb_img` con `mask`. Con `cast`, convertimos la máscara a un tipo compatible con `rgb_img` para poder realizar la operación:

```
masked_rgb_img = bsxfun(@times, rgb_img, cast(mask, 'like', rgb_img));
```

Luego realizamos la cuantificación sin dithering de manera análoga a apartados anteriores, y la mostramos en la primera posición del subplot:

```
[indexed_image_no_dither_masked, map_no_dither_masked] =  
rgb2ind(masked_rgb_img, 8, 'nodither');  
  
subplot(1, 2, 1);  
imshow(indexed_image_no_dither_masked, map_no_dither_masked);  
title('Imagen enmascarada cuantificada (3 bits) sin dithering');
```

Por último, cuantificamos la imagen con dithering, y la mostramos en la segunda posición del subplot para compararla con la primera:

```
[indexed_image_dither_masked, map_dither_masked] = rgb2ind(masked_rgb_img, 8,  
'dither');  
  
subplot(1, 2, 2);  
imshow(indexed_image_dither_masked, map_dither_masked);  
title('Imagen enmascarada cuantificada (3 bits) con dithering');
```

Al ejecutar el programa, obtenemos las siguientes imágenes:

Imagen enmascarada cuantificada (3 bits) sin dithering

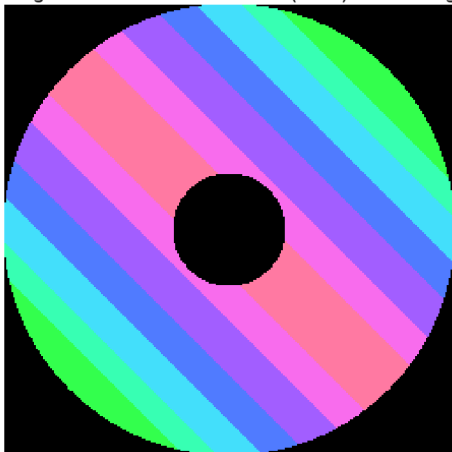


Imagen enmascarada cuantificada (3 bits) con dithering

